Do Target Wake Time and Uplink OFDMA Combine? An Offline Reinforcement Learning Study of Joint Scheduling in IEEE 802.11ax

Ahmed Selim Anwar, Ahmed Maksud, Joarder Jafor Sadique,
Charles Wiame, and Marcelo Menezes De Carvalho
Department of Electrical Engineering, *Texas State University*, San Marcos, TX, USA
Emails: {ahmed.selim.anwar, poy18, jjsadique, cwiame, mmcarvalho}@txstate.edu

***Abstract*—IEEE 802.11ax introduces Target Wake Time (TWT) and uplink OFDMA to reduce station energy in dense Wi-Fi deployments. The two mechanisms are usually configured separately, and no study measures station energy when an access point controls both. The difficulty is that a controller must adapt to changing traffic, yet online learning is impractical in a production network, and a hand-tuned rule needs an expert and is unlikely to be optimal. Offline reinforcement learning avoids both: a policy is trained from logged data alone, with no exploration on the live network. We cast joint control as a 760-way action, selecting a TWT schedule, a station-to-group assignment, and whether uplink OFDMA is enabled at each decision, and train five offline algorithms on 100,235 logged transitions in ns-3.44. Implicit Q-Learning reaches 107.3 J against a tuned hand-designed controller's 166.5 J, a $35.6\,\%$ reduction at higher throughput, and does so consistently: the spread over ten training runs is 2.6 J. Four of the five methods lose to that controller, and the energy of all five is ordered by how often each enables uplink OFDMA.**

***Index Terms*—IEEE 802.11ax, Target Wake Time, uplink OFDMA, offline reinforcement learning, energy efficiency, conservative Q-learning, off-policy evaluation.**

I. Introduction

IEEE 802.11ax gives an access point two ways to cut the energy cost of battery-powered stations. Target Wake Time (TWT) schedules when each station wakes, so radios sleep between service periods [1], [2]. Uplink OFDMA replaces contention with scheduled resource units, so several stations transmit at once [3], [4]. Both reduce energy, and recent work treats their configuration as a scheduling problem of its own [5], [6].

The two mechanisms are normally configured in isolation. TWT studies fix the OFDMA setting, and OFDMA studies fix or ignore the sleep schedule [7], [8]. Yet they compete for the same resource: a trigger frame is only useful to a station awake to receive it, so every OFDMA round draws on the airtime and the radio states TWT is trying to conserve.

No single setting serves a whole cell. A basic service set carries mixed traffic—latency-sensitive video, periodic telemetry and voice together [9], [10]—and a schedule tuned for light telemetry is wasteful under a video burst. A controller must adapt, which raises how it should be obtained.

A hand-coded rule is cheap to run but needs an expert to tune, and is unlikely to stay optimal away from the conditions it was tuned for. An idealised online learner could in principle track the optimum instant by instant, but it must explore on the live network, and exploration costs latency and risks service degradation [11], [12].

Offline reinforcement learning avoids both costs [13]. A policy is trained from an already-collected dataset and never acts on the network while learning, so it needs neither exploration nor expert-labelled actions—though the state features, reward and action catalogue remain design choices. The price is distribution shift, for which conservative value estimation is the standard remedy [14], [15]. Its wireless applications concentrate on cellular resource management [12], [16], leaving Wi-Fi power save largely unexamined.

We apply it to joint TWT and OFDMA control. Once per decision window the access point selects a TWT schedule, a station-to-group assignment and whether uplink OFDMA is enabled—a 760-way discrete action. Making the OFDMA bit part of the action rather than a property of the run is what makes the problem joint. Five offline algorithms are trained on $100{,}235$ transitions logged in ns-3.44 and compared against hand-designed and non-adaptive baselines.

Our contributions are the following.[1]

- A policy learned without online interaction or expert-set thresholds that cuts station energy against a tuned hand-designed controller at higher throughput, holding across ten training runs and against every fixed schedule tested—its advantage coming from switching schedules, not from finding a better static one.
- Evidence that at this operating point enabling the uplink scheduler raises energy at every TWT configuration tested, with five independently trained policies ordering by energy as they order by how often each enables it.

[1]Code, the ns-3 scenario and the frozen results that generate every table and figure: https://github.com/shuvo102459/802.11ax-OfflineRL-TWT-OFDMA

- A service-period-aligned trigger scheduler that solicits a station only while its service period can contain the whole exchange, removing a race in which a solicited station sleeps before replying and halving the measured cost of uplink OFDMA.

II. Related Work

TWT scheduling. TWT has been studied for IoT power saving [1], category-aware grouping [2], latency-sensitive traffic [9]; recent work treats the schedule itself as the object of optimisation [5]. Karaca [7] optimises the TWT mechanism jointly with scheduling, and Roy et al. [17] maximise timely throughput under TWT, allocating OFDMA resources *within* an awake group. Both fix a policy by analysis or optimisation. We fit one to logged data instead.

Uplink OFDMA scheduling. A separate line schedules OFDMA without TWT: deep RL for the OFDMA MAC [3], coexisting latency classes [10], hierarchical RL for joint uplink OFDMA and MU-MIMO [8], multi-agent RL for uplink OFDMA [4], [6]. Objectives are mostly throughput or delay, though some include power [6]; what none makes part of the control problem is station sleep. Both mechanisms do appear together [7], [17]; what is absent is a station-energy account in which the OFDMA decision is controlled per decision rather than assumed available, and in which sleeping more competes in the same action space.

Machine learning for Wi-Fi and radio resource management. Online RL has been applied to contention-window control [11] and to the problems above, with live-network exploration the obstacle to deployment. Offline RL [13] removes it and pays in distribution shift, addressed by conservative value estimation [14], [15]; its wireless applications concentrate on cellular resource management [12], [16], where action spaces are modest. We combine offline learning with per-decision control of the scheduler and score it on station energy. Maksud and Carvalho [18], coauthors here, apply *online* deep RL to dynamic TWT scheduling and share the ns-3 scenario and energy accounting; this paper adds the OFDMA bit and the grouping to the action, learns offline, and asks whether the result beats a tuned hand-designed controller.

III. System Model

A. Network and traffic

We consider a single basic service set: one access point and $N = 16$ stations on a 20 MHz 802.11ax channel, with a beacon interval of $T_{\mathrm{BI}} = 102.4$ ms. All traffic is uplink.

Stations belong to four device classes, four each, representing mixed traffic: a low-rate IoT sensor, a video camera, a voice assistant and a video streamer, offering 0.058, 2.037, 0.030 and 3.400 Mb/s respectively for 22.1 Mb/s in aggregate. The access point acts once every $\Delta = 20$ beacon intervals, or 2.048 s: 42 decisions per episode.

B. Target Wake Time

A TWT schedule partitions the beacon interval into K service periods of durations $\{w_1, \ldots, w_K\}$. Each station is assigned to one group, and group k is awake from $\sum_{j<k} w_j$ to $\sum_{j\le k} w_j$ in every beacon interval and sleeps outside it, so staggering trades contention against window length. The catalogue holds 20 schedules spanning one to four service periods and 20–90 ms of wake time, with 19 rules mapping the 16 stations to groups. One-group schedules make several rules coincide, so the 760 encoded actions realise fewer distinct configurations.

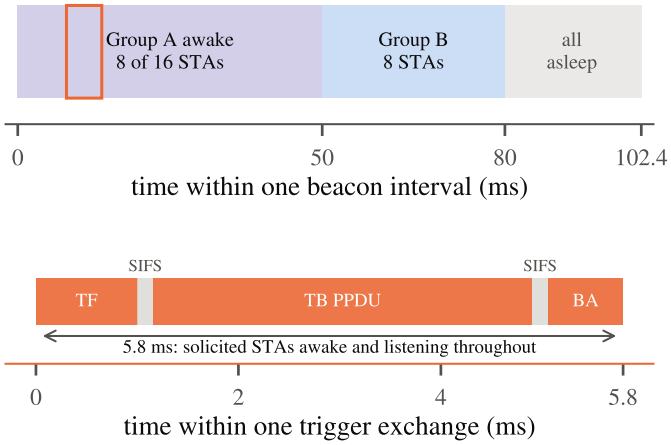


Fig. 1. *Top:* one beacon interval under a two-group TWT schedule. *Bottom:* a trigger exchange on its own scale. A solicited station is awake for the whole exchange—receiving, then transmitting its own reply—and the scheduler reserves the 5.8 ms worst case before admitting one, which is what makes uplink OFDMA compete with the schedule.

C. Uplink OFDMA

When uplink OFDMA is enabled the access point sends a trigger frame naming a set of stations and the resource unit each may use; those stations reply simultaneously in a trigger-based PPDU. Contention is replaced by scheduling at a cost: a solicited station is awake for the whole exchange. A station outside its service period cannot be solicited at all, so OFDMA multiplexes only *within* the awake group while consuming part of the window the schedule was keeping short. The scheduler solicits at most four stations per trigger, ns-3's default, against the nine 26-tone resource units a 20 MHz channel admits; Section VI reports both.

D. Control problem

At decision t the access point observes a state s_t and selects

$$a_t = b_t \cdot 380 + s_t^{\mathrm{sched}} \cdot 19 + g_t, \tag{1}$$

with $s^{\mathrm{sched}} \in \{0, \ldots, 19\}$ a schedule, $g \in \{0, \ldots, 18\}$ an assignment and $b \in \{0, 1\}$ enabling uplink OFDMA for that window: 760 joint actions. The objective is to minimise station energy without reducing delivered throughput.

IV. Methodology

A. State and reward

The state is $2 + 16 \times 8 = 130$-dimensional: total throughput and a proportional-fairness index, then per station throughput,

normalised backlog, three buffer-occupancy indicators, energy efficiency, drops and energy drawn. Rates and indicators are window statistics, counts are deltas over the preceding window. No clock is included, though backlog correlates with elapsed time.

The reward combines a per-station term and a fairness term. Each station contributes r_i, a weighted sum of $\tanh$-scaled throughput, energy efficiency and energy, a bonus for low buffer occupancy and a duty-cycle penalty; $r_{\text{sum}} = \sum_i r_i$ over stations active in the window and r_{fair} is Jain's index over $\{r_i\}$. Energy enters separately and explicitly, replacing the saturated term we inherited:

$$r = 0.30\, z\left(-\tanh \frac{E}{E^*}\right) + 0.55\, z(r_{\text{sum}}) + 0.15\, z(r_{\text{fair}}), \quad (2)$$

where E is the per-decision energy summed over stations, $E^* = 8000\,\text{mJ}$, and $z(\cdot)$ subtracts the mean and divides by the standard deviation of that component on the training split alone. This is a weighted surrogate, not a constrained optimisation: throughput is an evaluation criterion, not a guaranteed floor on the learned policy.

B. Dataset and training

We log 2,404 episodes, 100,235 transitions (Alg. 1). The ten-decision hold is deliberate: with a fresh draw each decision, measured energy is flat against schedule tightness—only a sustained schedule makes the gradient visible. Uniform sampling gives broad nominal coverage but does not concentrate data on high-performing trajectories—why behaviour cloning finishes last here. Splits are by episode, never by transition: 2,043 training, 361 held out. Episodes may truncate when the simulator ends a run—83 of 2,404 carry fewer than 42 transitions—and truncated states are bootstrapped, not treated as terminal. We train behaviour cloning, DQN+CQL, Double DQN, dueling DQN+CQL and IQL [14], [15], [19] at ten initialisations each. Ours is the discrete-action IQL: expectile regression for V_ψ and the SARSA-style target $r + \gamma V_\psi(s')$ for Q_θ as in [15], but with the policy extracted by $\arg\max_a Q$ rather than by advantage-weighted cloning, which a finite action set makes unnecessary.

C. Checkpoint selection

Choosing which checkpoint to deploy is harder than it looks when the network is off limits. Importance sampling is weak: the logger is uniform only at the ten-step refresh points, so the per-transition propensity is not $1/760$ and a match count does not characterise the effective sample size. Either way the overlap with a deterministic policy is small—tens of matching pairs among 15,025 held-out transitions. We rank by fitted Q evaluation [20] on held-out episode starts and validate that ranking against the simulator.

D. Service-period-aligned triggering

A trigger exchange started near the end of a service period cannot finish inside it. The station sleeps before it replies, and the access point is left awaiting a reply from a radio that has switched off. That is a protocol fault, not a missed transmission opportunity.

We therefore solicit a station only while its service period still has room for the entire exchange, which we take as 5.8 ms, covering the trigger, two short interframe spaces, a maximum-duration trigger-based PPDU and the acknowledgement. This aligns uplink OFDMA with the awake window by construction.

Algorithm 1 Offline joint TWT/OFDMA control

1: **Collect** $\mathcal{D}$: for each episode, draw $a \sim \text{Unif}(760)$, hold it for $H = 10$ decisions, log (s, a, r, s')
2: **Split** $\mathcal{D}$ by episode into $\mathcal{D}_{\text{tr}}, \mathcal{D}_{\text{ho}}$
3: **for** each algorithm m and seed $\sigma \in \{42..51\}$ **do**
4: $\quad \theta_{m,\sigma} \leftarrow$ train m on $\mathcal{D}_{\text{tr}}$, snapshot every 50 epochs
5: $\quad k^\star \leftarrow \arg\max_k \text{FQE}(\theta^{(k)}_{m,\sigma}; \mathcal{D}_{\text{tr}}, \mathcal{D}_{\text{ho}})$ {no simulator}
6: **end for**
7: **Deploy** $\theta^{(k^\star)}$: each decision, take $a_t = \pi_m(s_t)$, decode $(b_t, s_t^{\text{sched}}, g_t)$, and if b_t solicit only stations whose service period can still hold a whole exchange {π_m: $\arg\max_a Q$ for the value-based methods, $\arg\max_a \text{softmax}(f_\phi(s))$ for BC}

V. Experimental Setup

A. Simulator and parameters

All experiments use ns-3.44 with the ns3-ai interface. Table I lists the key parameters, naming the sweep wherever one chose the value; the full configuration ships with the artefact.

B. Seeds and protocol

Three disjoint seed sets are used, so no policy is evaluated on data that informed it: collection from 30000, tuning on 80000–80014, results on 90000–90049.

The fifty evaluation seeds are *paired*: a seed fixes the traffic, the time of any load change and its multiplier, so every policy meets an identical environment. Pairing reduces the variance from those shared conditions, which lets fifty seeds resolve differences that would otherwise need many more. Energy is reported over the 42-decision window, excluding the warm-up.

C. Baselines

Nine non-learned scenarios span the configuration space, from no power save to a tuned hand-designed controller (Table II).

The primary adaptive comparator is a hysteresis rule that raises the duty cycle when the observed backlog proxy crosses an upper threshold and lowers it below a lower one. Its thresholds, schedule pair and OFDMA mode come from a staged coordinate search on the tuning seeds, minimising energy subject to a service floor: mean throughput at least 95 % of the highest any candidate in that sweep achieves, so it cannot win by refusing traffic. The static baseline is chosen the same way.

TABLE I
SIMULATION AND TRAINING CONFIGURATION. EVERY VALUE IS THE ONE USED TO PRODUCE THE REPORTED RESULTS; WHERE A SWEEP CHOSE IT, THE SWEEP IS NAMED.

Scenario (ns-3.44)		
stations	16	four device classes, four each
channel	20 MHz	802.11ax, single BSS
STAs per trigger	4	ns-3 default; 20 MHz fits 9
CCA-busy current	273 mA	ns-3 default; TX/RX 232/66
beacon interval	102.4 ms	
decision interval Δ	20 BI = 2.048 s	42 decisions per episode
episode length	86 s	
offered load	0.058/2.037/ 0.030/3.400 Mb/s	IoT / camera / voice / streamer; 22.1 Mb/s total
load step	$\times\{1.5, 2.0, 2.5\}$	in 60 % of episodes, at $U[30, 70]$ s; seed-determined
Dataset		
transitions	100,235	2,404 episodes
split	85,210 / 15,025	by episode, 2,043 / 361
behaviour policy	uniform over 760	all 760 actions present
Training (all five algorithms)		
network	130–128–128–760	ReLU; dueling splits the head
optimiser	Adam, lr 10^{-3}	defaults otherwise
batch size	512	149 steps per epoch
epochs	200	milestone every 50
discount γ	0.95	
Bellman target clip	± 50	chosen so $\lvert V\rvert$ = 23–35 fits
CQL coefficient α	0.5	from an FQE sweep over $\{0.5, 1.0, 2.0\}$
IQL expectile	0.7	
training seeds	42–51	ten independent fits per algorithm (Table II)
Selection and evaluation		
model selection	FQE, 3×10^4 iters	episode-disjoint holdout
convergence gate	none	see Section VI
evaluation seeds	50 shared	disjoint from training
heuristic tuning seeds	15	80000–80014, disjoint

VI. RESULTS

A. The two mechanisms do not combine

Table II holds three pairs differing only in the OFDMA bit, and each time enabling it costs energy: $+62.8\,\%$ under the static schedule, $+33.6\,\%$ under random TWT, $+8.3\,\%$ with TWT off.

Throughput falls in all three, so this is not a throughput-for-energy trade.

The penalty is largest where TWT sleeps hardest and smallest where it does not sleep at all: soliciting a station competes with the schedule trying to keep it asleep.

B. The cost is carrier sensing, not lost sleep

Our first explanation was that triggers wake sleeping stations. The traces refute it. Mean sleep occupancy—the fraction of the control window a station spends in PHY sleep, averaged over the sixteen—differs by under 0.8 percentage points between the OFDMA arms. Stations sleep just as much; their awake time simply costs more.

Per-state traces from a separate instrumented run—the tuned controller with and without the scheduler, which differ by 113.7 J there—place $69\,\%$ of the extra energy in carrier-sense busy and $24\,\%$ in receive, $9\,\%$ in transmit, against $-1\,\%$ in idle. A solicited station is awake across the trigger, its own transmission and the acknowledgement, and the awake portion outside its own transmission is charged at the CCA-busy current.

The scenario leaves CCA-busy at ns-3's default 273 mA against its own 66 mA receive current, so we re-evaluated every load-bearing arm at 66 mA with the same policies and seeds. Energies fall 21–27 % unevenly, yet both conclusions hold: OFDMA costs $+57.8\,\%$ rather than $+62.8\,\%$, and IQL beats the controller by $31.2\,\%$ rather than $35.6\,\%$. Signs survive the power profile; magnitudes do not.

C. One algorithm beats the tuned rule

Given a free choice the tuned controller disables OFDMA, worth 99 J (265.5 against 166.5 J); we take that stronger form as the baseline.

IQL reaches 107.3 ± 2.6 J (Fig. 2a), the spread being the standard deviation over ten training runs, not a confidence interval. That is a $35.6\,\%$ reduction; the interval $[-36.5, -34.7]\,\%$ is a paired bootstrap over the fifty shared seeds of the per-seed mean across runs, and $p = 1.8 \times 10^{-15}$ is a two-sided Wilcoxon signed-rank test on the same pairing. Neither propagates uncertainty from resampling training runs; the across-run spread is reported separately. The reduction comes with higher mean throughput, 20.76 against 19.22 Mb/s, and a backlog proxy lower by 83 ms ($p = 9.8 \times 10^{-9}$ paired); the change in the fraction of decisions above three seconds is not statistically detectable ($p = 0.07$). Its worst fit of ten, 112.4 J, still beats every other method's best.

That column is a drain-time proxy—queue bytes over offered rate—not a measured latency, so it tracks standing backlog: flat across all 42 decisions, 2.6–2.9 s where throughput stays near offered load and 3.1–3.2 s where it does not. We use it as a guard against starving traffic, not as a latency claim.

D. Four of five do not, and selection is why

Table II is why we claim this for IQL and not for offline RL as a family. The *mean over runs* column averages the ten fits per evaluation seed, estimating expected performance over training randomness: DQN+CQL is $15.9\,\%$ worse than the hand-designed rule ($p = 8.6 \times 10^{-5}$), dueling DQN+CQL $16.6\,\%$ worse ($p = 4.3 \times 10^{-6}$), DDQN and behaviour cloning worse still.

The *oracle* column takes each method's lowest-energy fit, chosen on the same seeds it is then scored on. There the CQL variants sit within $1.6\,\%$ of the baseline with no difference detected ($p = 0.74$, 0.55)—a failure to detect, not evidence of equivalence, under retrospective selection. The gap between the columns is what matters: expected performance over training randomness is $16\,\%$ worse than a hand-written rule, while a retrospectively chosen fit is not distinguishable from it. Whether an off-policy estimator can pick that fit in advance across independent runs is open; we validate FQE only for ranking checkpoints within a run.

TABLE II

EVERY SCENARIO ON THE SAME 50 PAIRED EVALUATION SEEDS, ONE SIMULATOR AND ONE 42-DECISION MEASUREMENT WINDOW. WHERE UPLINK OFDMA IS ENABLED THE SCHEDULER IS SERVICE-PERIOD ALIGNED AND THE SLEEP GUARD IS DISABLED, SO NO ROW DEPENDS ON THAT WORKAROUND. THE HAND-DESIGNED CONTROLLER IS TUNED ON DISJOINT SEEDS UNDER A SERVICE FLOOR; THE ROW MARKED *free choice* LETS THAT SEARCH PICK ITS OWN OFDMA MODE, AND IT PICKS OFF. LEARNED ROWS GIVE THE MEAN AND STANDARD DEVIATION OVER 10 TRAINING SEEDS. THE LAST THREE COLUMNS COMPARE EACH LEARNED METHOD AGAINST THE *free choice* CONTROLLER AT 166.5 J, WILCOXON SIGNED-RANK: *mean over runs* AVERAGES THE TEN FITS PER SEED; *oracle* IS EACH METHOD'S LOWEST-ENERGY FIT, SELECTED ON THE SAME EVALUATION SEEDS IT IS SCORED ON, AND IS THEREFORE RETROSPECTIVE. *backlog* IS A DRAIN-TIME PROXY, PER-STATION QUEUE BYTES OVER THAT STATION'S OFFERED RATE, AVERAGED OVER STATIONS AND DECISIONS; IT IS NOT A MEASURED PACKET LATENCY. >3 S IS THE FRACTION OF DECISIONS WHOSE MEAN PROXY EXCEEDS THREE SECONDS. OFDMA: SHARE OF DECISIONS WITH THE SCHEDULER ENABLED.

						mean over runs		oracle
Scenario	E (J)	tput (Mb/s)	backlog (ms)	>3 s	OFDMA	Δ vs hand	p	Δ vs hand
No TWT, no OFDMA	441.1	24.95	2637	0.24	0 %			
Static TWT	173.3	21.03	2789	0.37	0 %			
Static TWT + OFDMA	282.2	19.82	2823	0.40	100 %			
Random TWT	171.1	14.47	3159	0.63	0 %			
Random TWT + OFDMA	228.5	13.51	3188	0.66	100 %			
OFDMA only, no TWT	477.8	24.30	2652	0.25	100 %			
Random joint action	198.8	14.29	3157	0.63	48 %			
Hand-designed, OFDMA on	265.5	18.37	2904	0.43	100 %			
Hand-designed, *free choice*	166.5	19.22	2888	0.41	0 %			
offline RL, 10 training seeds each								
IQL	107.3 ± 2.6	20.76	2805	0.38	0 %	−35.6 %	10^{-15}	−37.2 %
DQN+CQL	193.0 ± 10.1	20.54	2829	0.40	23 %	+15.9 %	10^{-5}	+1.6 %
Dueling+CQL	194.1 ± 14.7	20.49	2834	0.40	25 %	+16.6 %	10^{-6}	+0.7 %
DDQN	222.7 ± 13.5	15.64	3076	0.58	43 %	+33.8 %	10^{-15}	+23.7 %
BC	255.1 ± 12.4	19.78	2875	0.43	48 %	+53.2 %	10^{-15}	+42.7 %

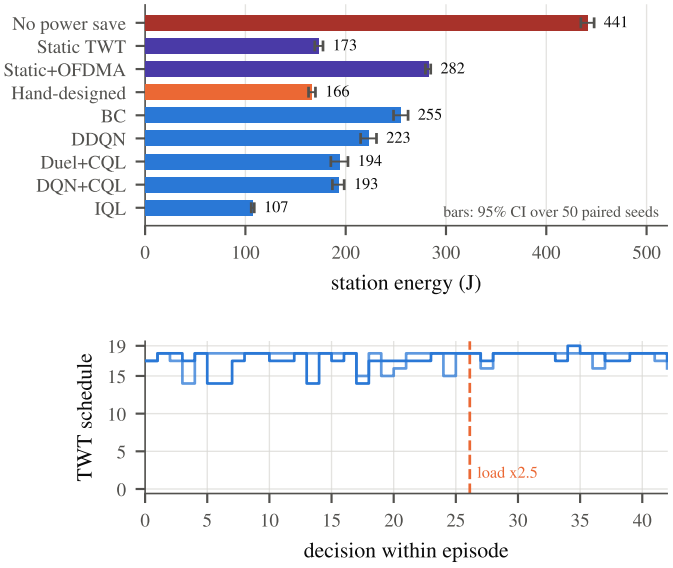


Fig. 2. *(a)* Mean station energy per scenario over the 50 paired seeds, 95 % bootstrap intervals, learned rows averaging ten training runs. *(b)* The schedule IQL selects at each of the 42 decisions of a stepped-load episode; dashed line, the load change.

IQL is the exception because it barely varies: its across-fit standard deviation is 2.6 J against 10.1 to 14.7 J for the others, so which fit you deploy does not matter.

E. What separates the methods is the OFDMA bit

The five methods order by energy as they order by how often they enable the scheduler, from IQL at 0 % to behaviour cloning at 48 %—the last because the logger chose the bit by coin flip and imitating it reproduces that.

The lowest-energy method is the one that disables the scheduler, and the coordinate search reaches the same configuration independently. The obvious objection is that we crippled the scheduler: ns-3 solicits four stations per trigger by default, while schedule 11 leaves eight awake and the channel admits nine resource units, so half the awake set was unreachable. Raising the limit to nine makes uplink OFDMA worse, not better—+87.4 % against the no-OFDMA arm instead of +62.8 % under the static schedule, and +83.3 % instead of +59.5 % for the hand-designed controller, both +15 % on the default at $p = 2 \times 10^{-15}$ with throughput falling at each step. Solicitation cost scales with the stations a trigger names; the benefit scales with the data they have, and in this mix they have little.

Each seed either holds its offered load constant or steps it by 1.5 to 2.5× partway through. Splitting the fifty on that flag gives 35.1 % below the baseline on the 19 constant-load seeds ($p = 3.8 \times 10^{-6}$) and 35.8 % on the 31 stepped ones ($p = 9.3 \times 10^{-10}$): robustness to within-distribution load variation, not transfer to unseen traffic.

F. Is the adaptation necessary?

IQL and the tuned controller both disable uplink OFDMA, so the gap between them cannot come from that bit. Would one well-chosen fixed configuration do as well? We swept all 20 schedules at the deployed grouping on the tuning seeds with OFDMA off, picked the Pareto-efficient ones from tuning data alone, and evaluated those on the untouched evaluation seeds.

None matches IQL. At its energy the best fixed schedule gives 12.62 Mb/s against 20.76—64 % more data for the same power, the energy difference undetectable ($p = 0.12$); at its throughput the best costs 179.1 J against 107.3. One is beaten on both axes at once (145.4 J, 17.93 Mb/s), and on tuning seeds none of the twenty comes within 10 % of IQL's energy at 90 % of its throughput.

The traces say why (Fig. 2b). IQL uses 14.9 distinct actions per 42-decision episode, and the schedules it favours are poor alone: schedule 18 (43 % of decisions) and schedule 19 (23 %) give 15.64 and 12.74 Mb/s as fixed policies. Holding either costs about what IQL costs while moving a third less data, so the gain is in moving between schedules. We swept schedules at the deployed grouping only, so this is the best fixed *schedule*, not of all 380 configurations.

G. Validation

Dataset and reported runs do not share identical settings. Collection predates service-period alignment, using naive triggering and the 8 ms deferral, while every arm reported here is aligned with the deferral off. The deferral is bypassed when the OFDMA bit is clear, so the two coincide on OFDMA-off transitions and differ only where the logger set the bit—there training charges OFDMA more than deployment does, biasing a learner toward disabling it. Table II is unaffected: every arm ran aligned.

Four instrumentation faults materially affected earlier evaluations, each invisible in aggregate statistics: a convergence gate that discarded a sound checkpoint ranking (6.5 % energy), a saturated reward energy term (13.3 J), mismatched measurement windows, and the deferral above (+62.0 % alone, yet moving sleep *occupancy* under 0.8 %). Each is quantified in the artefact.

VII. Conclusion

TWT scheduling, station grouping and uplink OFDMA enablement form one 760-way offline control problem. We trained five offline algorithms on it and measured them against nine non-learned scenarios on fifty paired seeds.

In this setting IQL reduces station energy by 35.6 % against a tuned hand-designed controller at higher throughput, consistently over ten training runs, and no fixed schedule tested reaches its operating point. The result is about IQL rather than offline RL: four of five methods lose to that controller, and the two CQL variants match it only under retrospective selection, so checkpoint selection rather than policy optimisation is the open difficulty. Under the same conditions, enabling the evaluated uplink scheduler raised energy at every TWT configuration tested, and both a learned policy and an independent coordinate search disable it when free to choose.

Four limits bound these claims. The topology is one basic service set, sixteen stations and 20 MHz, soliciting at most four stations per trigger against nine resource units; we tested four and nine, not beyond. Station mix and traffic model are identical across every arm, so comparisons are internally controlled but not shown to transfer. The energy model leaves CCA-busy at ns-3's default 273 mA against its own 66 mA receive current; re-evaluating at 66 mA moves energies 21–27 % unevenly yet changes no ordering. Finally the logged actions were held for ten decisions and collected before service-period alignment, so training and deployment differ on the OFDMA-on half of the data.

Higher station counts, wider channels and larger solicitation sets could still reverse the substitution; testing that needs a sourced power profile and a sweep of the grouping dimension.

References

[1] Q. Chen, Z. Weng, X. Xu, and G. Chen, "A target wake time scheduling scheme for uplink multiuser transmission in IEEE 802.11ax-based next generation WLANs," *IEEE Access*, vol. 7, pp. 158 207–158 222, 2019.

[2] W. Qiu, G. Chen, K. N. Nguyen, A. Sehgal, P. Nayak, and J. Choi, "Category-based 802.11ax target wake time solution," *IEEE Access*, vol. 9, pp. 100 154–100 172, 2021.

[3] D. Kotagiri, K. Nihei, and T. Li, "Distributed convolutional deep reinforcement learning based OFDMA MAC for 802.11ax," in *IEEE ICC*, 2021, pp. 1–6.

[4] M. Han, X. Sun, W. Zhan, Y. Gao, and Y. Jiang, "Multi-agent RL based uplink OFDMA for IEEE 802.11ax networks," *IEEE Trans. Wireless Commun.*, vol. 23, no. 8, pp. 8868–8882, 2024.

[5] F. Busacca, C. Puligheddu, F. Raviglione, R. Rusca, C. Casetti, C. F. Chiasserini, and S. Palazzo, "Target wake time scheduling for time-sensitive and energy-efficient Wi-Fi networks," *IEEE Trans. Mobile Comput.*, vol. 25, no. 3, 2026.

[6] Y. Liu *et al.*, "Priority-aware resource allocation in AoI-oriented UL-OFDMA Wi-Fi networks based on multiagent reinforcement learning," *IEEE Internet Things J.*, 2025.

[7] M. Karaca, "Joint optimization of TWT mechanism and scheduling for IEEE 802.11ax," arXiv:2006.02235, 2020.

[8] H. Noh, H. Lee, and H. J. Yang, "Joint uplink OFDMA and MU-MIMO for IEEE 802.11ax: Deep hierarchical RL," *IEEE Commun. Lett.*, vol. 28, no. 8, pp. 1800–1804, 2024.

[9] C. Yang, J. Lee, and S. Bahk, "Target wake time scheduling strategies for uplink transmission in IEEE 802.11ax networks," in *IEEE WCNC*, 2021, pp. 1–6.

[10] Q. Tan, J. He, and Y. Gao, "Deep reinforcement learning based OFDMA scheduling for WiFi networks with coexisting latency-sensitive and high-throughput services," in *ICTC*, 2024, pp. 146–150.

[11] W. Wydmański and S. Szott, "Contention window optimization in IEEE 802.11ax networks with deep reinforcement learning," in *IEEE WCNC*, 2021, pp. 1–6.

[12] K. Yang, C. Shi, C. Shen, J. Yang, S.-P. Yeh, and J. J. Sydir, "Offline RL for wireless network optimization with mixture datasets," *IEEE Trans. Wireless Commun.*, vol. 23, no. 10, pp. 12 703–12 716, 2024.

[13] S. Levine, A. Kumar, G. Tucker, and J. Fu, "Offline reinforcement learning: Tutorial, review, and perspectives on open problems," arXiv:2005.01643, 2020.

[14] A. Kumar, A. Zhou, G. Tucker, and S. Levine, "Conservative Q-Learning for offline reinforcement learning," in *NeurIPS*, 2020, pp. 1179–1191.

[15] I. Kostrikov, A. Nair, and S. Levine, "Offline reinforcement learning with implicit Q-Learning," in *ICLR*, 2022.

[16] E. Eldeeb and H. Alves, "An offline multi-agent reinforcement learning framework for radio resource management," *IEEE Trans. Mobile Comput.*, 2026, arXiv:2501.12991.

[17] R. Roy, R. V. Bhat, P. Hathi, N. Akhtar, and N. M. Balasubramanya, "Maximization of timely throughput with target wake time in IEEE 802.11ax," in *IEEE ICC*, 2023, pp. 647–652.

[18] A. Maksud and M. M. Carvalho, "Deep reinforcement learning-based dynamic TWT scheduling for heterogeneous Wi-Fi networks," in *ICCCN*, 2026.

[19] H. van Hasselt, A. Guez, and D. Silver, "Deep reinforcement learning with double Q-learning," in *AAAI*, 2016, pp. 2094–2100.

[20] H. M. Le, C. Voloshin, and Y. Yue, "Batch policy learning under constraints," in *ICML*, 2019, pp. 3703–3712.